

# CSE 717 — Information Security

## Topic 14: Digital Signature — Generic Model, Properties, Vulnerabilities

Complete Past Paper Solutions: 2020–2024

Source: general security knowledge (Stallings, Digital Signatures) — partial Lc#2 overlap (hash properties); diagram is a no-source cheat sheet

**Every Topic-14 question 2020–2024 included. Zero omissions.**

## Contents

---

|          |                                                                                                                                               |          |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>2024 — Q2(a): Digital Signature + Vulnerable Attacks (3); Section B Q2(c): Same Theme (3); Section B Q8(d): Generic Model Diagram (1)</b>  | <b>5</b> |
| <b>2</b> | <b>2023 — Q2(a): Digital Signature + Vulnerable Attacks (3); Section B Q5(c): Attack Types (3); Section B Q5(d): RSA Key-Reuse Safety (1)</b> | <b>6</b> |
| 2.1      | Parts (a) and Section B (c): Digital Signature + Attacks . . . . .                                                                            | 6        |
| 2.2      | Part Section B (d): RSA Key-Reuse Safety After Private-Key Leak . . . . .                                                                     | 6        |
| <b>3</b> | <b>2022 — Section B Q5(b): Properties + Scheme Requirements; Q7(a): Signature vs Confidentiality Order + Threats to Direct DS</b>             | <b>7</b> |
| 3.1      | Part (a): Properties + Scheme Requirements . . . . .                                                                                          | 7        |
| 3.2      | Part (b): Signature vs Confidentiality Order + Threats to Direct Digital Signature . . . . .                                                  | 7        |
| <b>4</b> | <b>2021 — Section B Q2(c): Generic Model Diagram (2 marks)</b>                                                                                | <b>8</b> |
| <b>5</b> | <b>2020 — Q7(a): Generic Model of Digital Signature Process (4.75 marks)</b>                                                                  | <b>9</b> |

## Quick Reference — The 5 Recurring Sub-Patterns (CHEAT SHEET)

**Diagram-heavy, low-prerequisite, CONFIRMED EVERY YEAR (5/5, 1–7 marks)**

Two recurring forms: (1) **draw the generic model** of the digital signature process (1–4.75 marks, pure diagram), and (2) **conceptual** questions on properties/attacks/requirements (3 marks). Combine with Topic 13 cheat sheet — both are guaranteed, low-effort marks. 30min investment.

### 1. What Is a Digital Signature?

A **digital signature** is a cryptographic value, computed from a message and the sender's **private key**, that is attached to the message to provide:

- **Authentication** — proves the message came from the claimed sender.
- **Integrity** — proves the message was not altered in transit.
- **Non-repudiation** — the sender cannot later deny having sent the message.

**Key idea:** the signer's private key is used to “sign” (encrypt a hash of the message), and anyone holding the corresponding **public key** can verify the signature — but only the signer could have produced it.

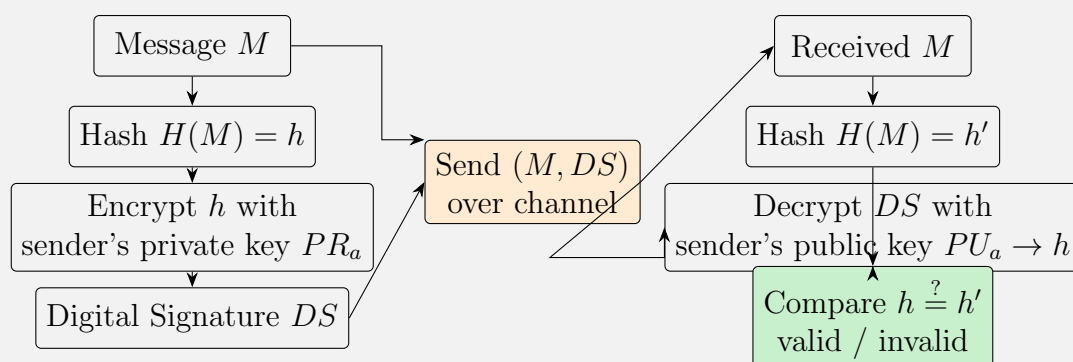
### 2. Generic Model of the Digital Signature Process

**Sender side:**

1. Compute a hash of the message:  $h = H(M)$ .
2. Encrypt the hash with the sender's **private key**  $PR_a$ :  $DS = E_{PR_a}(h)$ . This  $DS$  is the digital signature.
3. Send  $(M, DS)$  to the receiver.

**Receiver side:**

1. Decrypt  $DS$  using the sender's **public key**  $PU_a$  to recover  $h$ :  $h = D_{PU_a}(DS)$ .
2. Independently compute  $h' = H(M)$  on the received message.
3. **Compare**  $h \stackrel{?}{=} h'$ : if equal  $\rightarrow$  signature valid (message authentic & unmodified); if not  $\rightarrow$  reject.



## Symbol Legend — What Each Term Means

| Symbol                 | Meaning                                                                           |
|------------------------|-----------------------------------------------------------------------------------|
| $M$                    | The original message (plaintext) being sent.                                      |
| $H(M) = h$             | Hash function applied to $M$ , producing digest $h$ .                             |
| $PR_a$                 | Sender A's <b>private key</b> (secret, only A has it).                            |
| $PU_a$                 | Sender A's <b>public key</b> (shared with everyone).                              |
| $E_{PR_a}(h)$          | Encrypt $h$ using A's private key $\rightarrow$ produces $DS$ .                   |
| $DS$                   | Digital Signature = the encrypted hash.                                           |
| $D_{PU_a}(DS)$         | Decrypt $DS$ using A's public key $\rightarrow$ recovers $h$ .                    |
| $h'$                   | Hash computed by the <b>receiver</b> , independently, from the received $M$ .     |
| $h \stackrel{?}{=} h'$ | The verification check: "does the decrypted hash match the freshly computed one?" |

Flow walkthrough (left  $\rightarrow$  right):

- **Sender column (top to bottom):** hash  $M$  to get  $h \rightarrow$  encrypt  $h$  with  $PR_a$  to get  $DS$ .
- **Orange box:** the pair  $(M, DS)$  is sent together over the channel — two arrows feed it (one from  $M$ , one from  $DS$ ).
- **Receiver column:** *two independent paths* from the orange box: (1)  $M$  arrives and is re-hashed to give  $h'$ ; (2)  $DS$  is decrypted with  $PU_a$  to recover  $h$ .
- **Green box:** compare  $h$  (recovered) with  $h'$  (freshly computed). Equal  $\rightarrow$  valid signature; not equal  $\rightarrow$  reject.

**Why this proves authenticity:** only A's private key  $PR_a$  could have produced a  $DS$  that decrypts correctly with A's public key  $PU_a$  into the matching hash  $h$ . If  $h = h'$ , the message is both *authentic* (came from A) and *unmodified* (matches  $h'$ ).

## 3. Properties / Requirements of a Digital Signature Scheme

## A digital signature must:

- **Verify the author** and the date/time of the signature.
- **Authenticate the contents** at the time of signing.
- Be **verifiable by third parties**, to resolve disputes.

## Practical requirements on the signature itself:

- Must be a bit pattern that **depends on the message** being signed.
- Must use information **unique to the sender** (the private key) — to prevent both forgery and denial.
- Must be **relatively easy to produce**.
- Must be **relatively easy to recognize and verify**.
- Must be **computationally infeasible to forge** — either (a) by constructing a new message for an existing signature, or (b) by constructing a fraudulent signature for a given message.
- Must be **practical to retain a copy** of the digital signature in [redacted]

#### 4. Attack / Forgery Types Vulnerable to Digital Signatures

- **Key-only attack:** attacker **only knows the signer's public key**.
- **Known-message attack:** attacker has a **set of previously signed messages**.
- **Chosen-message attack:** attacker can **choose messages and obtain valid signatures for them** (generic / directed / adaptive variants).

Resulting forgery outcomes:

- **Total break:** attacker **recovers the signer's private key**.
- **Universal forgery:** attacker can **forge a signature for any message**.
- **Selective forgery:** attacker can **forge a signature for one chosen message**.
- **Existential forgery:** attacker can **produce at least one valid (message, signature) pair not previously seen** — weakest but still a break of security.

#### 5. Direct Digital Signature: Order vs. Confidentiality + Threats

Order of signature vs. confidentiality function:

- **Sign-then-encrypt** (preferred): sign the plaintext message first, then encrypt *both* the message and its signature with a shared secret key. The signature is now protected inside the encrypted envelope, and disputes can later be resolved using the original (unencrypted) signed message.
- **Encrypt-then-sign** is generally discouraged: a third party verifying the signature would need access to the plaintext, weakening confidentiality.

**Threats to a *direct* digital signature scheme** (security depends entirely on the sender's private key):

1. **Key compromise (forgery):** if the sender's private key is actually stolen, the attacker can forge the sender's signature on new messages.
2. **Repudiation via false claim:** the sender can falsely claim their private key was lost/stolen at some earlier time, to disown a signature they actually created (denial of responsibility).

**Mitigation:** timestamps + a trusted third party (e.g. arbitration/notarization) to bind signatures to a verified time.

#### Bonus: RSA — Safety After a Private-Key Leak (2023 Section B Q5(d))

If a user's RSA **private key** is leaked:

- **Past encrypted messages are NOT safe** — an attacker who recorded old ciphertexts can now decrypt all of them with the leaked private key (RSA, as used here, provides **no forward secrecy**).
- **Past signatures become untrustworthy** — the attacker could forge new signatures claiming to be from a past date, and the legitimate signer can plausibly deny old signatures (repudiation risk).
- **Conclusion:** reusing the same RSA key pair indefinitely is **not safe** once compromised — keys must be revoked/rotated immediately, and a certificate revocation mechanism (CRL/OCSP) is needed.

2024 — Q2(a): Digital Signature + Vulnerable Attacks (3); Section B Q2(c): Same Theme (3); Section B Q8(d): Generic Model Diagram (1)

Question — 2024 Q2(a) and Section B Q2(c)

What is a digital signature? What types of attacks make digital signatures vulnerable? [3 + 3]

Answer — 2024 Q2(a) / Section B Q2(c)

**What is a digital signature:** see Cheat Sheet #1 — a value computed from the message and the sender's private key, attached to provide **authentication**, **integrity**, and **non-repudiation**.

**Attack types that make it vulnerable:** see Cheat Sheet #4:

- **Key-only**, **known-message**, and **chosen-message** attacks (generic/directed/adaptive).
- Resulting forgeries: **total break**, **universal**, **selective**, **existential forgery**.

Question — 2024 Section B Q8(d)

Draw the generic model of the digital signature process. [1]

Answer — 2024 Section B Q8(d)

See Cheat Sheet #2 diagram: **sender** hashes  $M \rightarrow h$ , encrypts  $h$  with private key  $PR_a$  to get  $DS$ , sends  $(M, DS)$ ; **receiver** decrypts  $DS$  with public key  $PU_a$  to recover  $h$ , independently hashes received  $M$  to get  $h'$ , and **compares**  $h$  vs  $h'$ .

## 2023 — Q2(a): Digital Signature + Vulnerable Attacks (3); Section B Q5(c): Attack Types (3); Section B Q5(d): RSA Key-Reuse Safety (1)

### Question — 2023 Q2(a) and Section B Q5(c,d)

- (a) What is a digital signature? What types of attacks make it vulnerable? [3]
- (b) (Section B Q5(c)) What attack types is a digital signature vulnerable to? [3]
- (c) (Section B Q5(d)) If a user's RSA private key is leaked, is reusing the same key pair safe? [1]

### Parts (a) and Section B (c): Digital Signature + Attacks

#### Answer — 2023 Q2(a) / Section B Q5(c)

Identical content to 2024 Q2(a) above — see Cheat Sheet #1 (definition: authentication, integrity, non-repudiation) and Cheat Sheet #4 (key-only / known-message / chosen-message attacks → total break / universal / selective / existential forgery).

### Part Section B (d): RSA Key-Reuse Safety After Private-Key Leak

#### Answer — 2023 Section B Q5(d)

See Cheat Sheet (Bonus): **NO** — a leaked RSA private key means:

- All **past ciphertexts** (recorded by an eavesdropper) can now be decrypted — **no forward secrecy**.
- All **past signatures** become questionable — attacker can forge new ones, legitimate owner can deny old ones.
- The key pair must be **revoked and rotated immediately**; continuing to use it is unsafe.

## 2022 — Section B Q5(b): Properties + Scheme Requirements; Q7(a): Signature vs Confidentiality Order + Threats to Direct DS

### Question — 2022 Section B Q5(b) and Q7(a)

- (a) (Section B Q5(b)) What properties should a digital signature have? What are the requirements for a digital signature scheme?
- (b) (Q7(a)) What is the correct order of applying the signature function and the confidentiality function? What are the threats to a direct digital signature scheme?

### Part (a): Properties + Scheme Requirements

#### Answer — 2022 Section B Q5(b)

See Cheat Sheet #3 in full:

- **Properties:** must verify author + date/time, authenticate contents at signing time, be verifiable by third parties to resolve disputes.
- **Scheme requirements:** signature depends on the message; uses sender-unique info (private key); easy to produce & verify; computationally infeasible to forge; practical to store.

### Part (b): Signature vs Confidentiality Order + Threats to Direct Digital Signature

#### Answer — 2022 Q7(a)

See Cheat Sheet #5:

- **Correct order: sign-then-encrypt** — sign the plaintext message, then encrypt the message+signature together with a shared secret key. (Encrypt-then-sign weakens confidentiality, since verifiers would need plaintext access.)
- **Threats to direct digital signature** (security rests entirely on the sender's private key):
  1. **Key compromise** — stolen private key lets an attacker forge the sender's signature.
  2. **False repudiation** — sender falsely claims the key was stolen earlier, to disown a genuine signature.

**Mitigation:** timestamps + a trusted third party (arbitration).

**2021 — Section B Q2(c): Generic Model Diagram (2 marks)****Question — 2021 Section B Q2(c)**

Draw the generic model of the digital signature process.

[2]

**Answer — 2021 Section B Q2(c)**

Same diagram as Cheat Sheet #2: **sender**  $M \xrightarrow{H} h \xrightarrow{\text{encrypt with } PR_a} DS$ , send  $(M, DS)$ ; **receiver** decrypts  $DS$  with  $PU_a \rightarrow h$ , hashes received  $M \rightarrow h'$ , **compares**  $h$  vs  $h'$  to validate.



**2020 — Q7(a): Generic Model of Digital Signature Process (4.75 marks)****Question — 2020 Q7(a)**

Draw and explain the generic model of the digital signature process.

[4.75]

**Answer — 2020 Q7(a)**

Same diagram + explanation as Cheat Sheet #2 — this is the highest-mark version (4.75), so write out **all 6 steps** explicitly (3 sender-side + 3 receiver-side):

1. Sender computes  $h = H(M)$ .
2. Sender encrypts  $h$  with private key  $PR_a$  to get  $DS$ .
3. Sender transmits  $(M, DS)$ .
4. Receiver decrypts  $DS$  with sender's public key  $PU_a$  to recover  $h$ .
5. Receiver independently computes  $h' = H(M)$  on the received message.
6. Receiver compares  $h$  vs  $h'$  — equal  $\Rightarrow$  authentic & unmodified; not equal  $\Rightarrow$  reject.

Plus the diagram from Cheat Sheet #2.

## Exam Strategy Summary

### High-Yield Checklist for Digital Signature (5/5 years, 1–7 marks — GUARANTEED)

1. **Generic model diagram** (sender hashes → encrypt with private key → DS; receiver decrypts with public key, re-hashes, compares) — asked EVERY year (2020, 2021, 2024) in some form. Practice drawing it from memory in under 2 minutes.
2. **Definition + vulnerable attack types** (key-only / known-message / chosen-message → total break / universal / selective / existential forgery) — asked 2023, 2024.
3. **Properties + scheme requirements** (verify author/date, authenticate contents, third-party verifiable; signature depends on message, uses private key, easy to produce/verify, infeasible to forge, practical to store) — asked 2022.
4. **Sign-then-encrypt order + threats to direct DS** (key compromise, false repudiation; mitigated by timestamps + trusted third party) — asked 2022.
5. **RSA private-key leak** ⇒ **no forward secrecy, revoke & rotate** — asked 2023, cheap 1 mark.

*Total: 30 minutes of focused memorization for 1–7 GUARANTEED marks every single year. Pairs naturally with Topic 13 (DES/Feistel) cheat sheet.*